Reviewer Pack — Minimal Rank of Geometric Entanglement over Monotone Circuit…

Entry b20fea6bc2b7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 07:47:59 UTC

Minimal Rank of Geometric Entanglement over Monotone Circuit Depth

Entry ID: `b20fea6bc2b7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 07:47:59 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Entanglement) **Field B** (complexity object): Complexity Theory: Monotone Circuit Depth

Statement:

> {'stmt1': 'For a monotone circuit C computing the k-CLIQUE function, the minimal rank of its quantum geometric entanglement is Θ(n^k).', 'stmt2': 'Quantum geometric entanglement measures the non-classical correlations in a quantum state.', 'stmt3': 'This conjecture implies that for large n, monotone circuits require an exponential number of bits to represent k-CLIQUE.'}

Rationale (proposer's reasoning):

> {'ration1': "The use of quantum information theory's geometric entanglement could reveal new insights into the structure of monotone circuits.", 'ration2': 'Geometric entanglement is known for capturing non-local correlations, which may be related to the complexity of circuit computations.', 'ration3': 'This conjecture builds on the connection between quantum phenomena and computational complexity.'}

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `af3ad03900abe8eb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 random seeds, the minimal rank of the quantum geometric entanglement for monotone circuits computing k-CLIQUE is within a factor of 2 from Θ(n^k), and no seed produces a rank greater than 2Θ(n^k). The conjecture is falsified if any seed yields a rank outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum geometric entanglement" AND "monotone circuit depth" - "k-CLIQUE function" AND minimal rank AND quantum information theory" - "exponential number of bits" AND monotone circuits AND geometric entanglement

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```
    def rank(A):
        A = [row[:] for row in A]
        r = gaussian_elimination(A)
        return sum(1 for row in r if any(row[j] != 0 for j in range(len(row))))

    n_values = [5, 10, 15, 20, 30, 40]
    k = 3
    max_rank = 0

    for n in n_values:
        instances_tested = 0
        total_rank = 0
        for _ in range(5):
            # Generate a random graph with n vertices and edges
            G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
            for i in range(n):
                G[i][i] = 0

            # Convert the graph to a quantum state matrix
            H = [[G[i][j] * (2 ** (-n)) for j in range(n)] for i in range(n)]

            # Compute the minimal rank of the quantum state
            current_rank = rank(H)
            total_rank += current_rank
            instances_tested += 1

        avg_rank = total_rank / instances_tested
        max_rank = max(max_rank, avg_rank)

    conjecture_holds = avg_rank <= 2 * n_values[-1] ** k and max_rank <= 2 * n_values[-1] ** k
    counterexample = f"avg_rank={avg_rank}, max_rank={max_rank}" if not conjecture_holds else ""

    return {
        "metric_name": "minimal_rank",
        "metric_value": avg_rank,
        "instances_tested": instances_tested * len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    avg_rank = sum(result["metric_value"] for result in results) / len(results)
    max_rank = max(result["metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
```

```
        print(f"RESULT: SUPPORTED mean={avg_rank} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and max_rank > 2 * n_values[-1]
        counterexample = f"avg_rank={avg_rank}, max_rank={max_rank}"
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_648db2f8.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_648db2f8.py", line 68, in run_trial
    current_rank = rank(H)
                   ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_648db2f8.py", line 48, in rank
    r = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_648db2f8.py", line 25, in gaussian_elim
    A[i], A[max_row] = A[max_row], A[i]
                       ~^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means the conjecture could not be evaluated according to the pre-registered support and falsification conditions. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11753
2	preregistration	ollama_remote	glm4:latest	0	0	5808
3	novelty	ollama_remote	glm4:latest	0	0	4663

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	7918
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16159
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10294
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8196
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11892
9	judge	ollama_remote	glm4:latest	0	0	12007

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 88689 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b20fea6bc2b7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b20fea6bc2b7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b20fea6bc2b7.tar.gz` (if generated)